

lisp_eval.c 深度解析

源自 Lisp 解释器项目 `cuzperf/lisp-interpreter` 的会话讲解

目录

1. [整体设计架构](#)
2. [1.1 值表示](#)
3. [1.2 运行时数据结构](#)
4. [1.3 内置函数枚举](#)
5. [1.4 顶层入口: `eval\_toplevel`](#)
6. [1.5 求值核心: `eval\_sexp`](#)
7. [核心机制详解](#)
8. [2.1 尾调用优化: `tail\_eval` 宏](#)
9. [2.2 尾上下文传递: `tail\_macro`](#)
10. [2.3 列表求值: `TYPE\_LIST` 分支](#)
11. [2.4 内建函数应用: `apply\_builtin`](#)
12. [2.5 用户函数应用: `apply` 分支](#)
13. [2.6 `end`: 与尾调用传播](#)
14. [辅助函数详解](#)
15. [3.1 `eqp`——相等判定](#)
16. [3.2 `copy\_body`——闭包变量捕获](#)
17. [3.3 `prepare\_env`——形参绑定](#)
18. [3.4 `eval\_sym`——符号求值](#)
19. [典型例子跟踪](#)
20. [4.1 定义函数: `\(def add \(fn \(x y\) \(+ x y\)\).\)`](#)
21. [4.2 调用函数: `\(add 3 4\)`](#)
22. [4.3 闭包捕获: `let` 展开后](#)
23. [4.4 宏展开](#)
24. [设计要点总结](#)

1. 整体设计架构

1.1 值表示

所有值都是 `uintptr_t`，用**低位标签**区分类型：

宏	值	含义
<code>TAG_NUM</code>	<code>0x0</code>	数字，直接编码在指针高位，零堆分配
<code>TAG_LIST</code>	<code>0x1</code>	cons 单元，堆分配
<code>TAG_SYM</code>	<code>0x2</code>	符号，堆分配
<code>TAG_OTHER</code>	<code>0x3</code>	内建函数等，堆分配， <code>type</code> 字段在结构体首字节

```
#define tag(x) ((x)&0x3)           // 取低 2 位标签
#define ptr(x) ((void*)((x) & ~(value_t)0x3)) // 取指针（清除标签）
#define tagptr(x, t) (((value_t)(x)) | (t))    // 指针 + 标签
#define number(x) (((value_t)(x)) << 2)       // 数字左移 2 位（低位留 0）
```

数字编码示例：`number(42) = (42 << 2) = 0xA8`，`tag = 0x0`

1.2 运行时数据结构

求值栈 (`g_stack / g_sp`): 求值过程中的临时值，约 160K

环境栈 (`g_env_stack / g_env_sp`): 词法环境，布局 [`val1, sym1, val2, sym2, ...`]

符号表 (`syntab`): 二叉查找树，存全局绑定

堆 (`g_heap`): cons 单元、符号等，含 Cheney 式 GC

环境栈结构示意：

```
索引:  0      1      2      3      4      5      ...
      val1 sym1 val2 sym2 val3 sym3 ...
```

查找时从 `g_env_sp - 1` 向下每两步一跳，实现词法作用域的内层优先。

1.3 内置函数枚举

`CL_BUILTIN_FUNCTIONS(XX)` 宏定义了两类内置，用 `BuiltinCode` 枚举：

```
#define CL_BUILTIN_FUNCTIONS(XX) \
    XX(B_FN,      "fn")      \
    XX(B_MACRO,   "macro")   \
    XX(B_QUOTE,    "quote")   \
    XX(B_COND,    "cond")    \
    XX(B_DO,      "do")      \
    XX(B_DEF,     "def")     \
    XX(B_AND,     "and")     \
    XX(B_OR,      "or")      \
    XX(F_ADD,     "+" )      \
    XX(F_SUB,     "-" )      \
    XX(F_DIV,     "/" )      \
    XX(F_MUL,     "*" )      \
    XX(F_LT,      "<" )      \
    XX(F_GT,      ">" )      \
    XX(F_EQ,      "=" )      \
    XX(F_NOT,     "not")     \
    XX(F_CONS,    "cons")    \
    XX(F_HEAD,    "head")    \
    XX(F_TAIL,    "tail")    \
    XX(F_EVAL,    "eval")    \
    XX(F_APPLY,   "apply")   \
    XX(F_LISTP,   "list?")   \
    XX(F_SYMBOLP, "symbol?") \
    XX(F_NUMBERP, "number?") \
    XX(F_BUILTINP, "builtin?") \
    XX(F_PRINT,   "print")   \
    XX(N_BUILTINS, "")       \
```

前缀	含义	求值策略	示例
B_	Builtin special form	参数 不求值 ，自行控制	B_FN 、 B_DEF 、 B_COND 、 B_QUOTE
F_	Function	参数 先全部求值 再使用	F_ADD 、 F_CONS 、 F_HEAD

1.4 顶层入口: `eval_toplevel`

用于加载文件时批量处理多个顶层表达式:

```

value_t eval_toplevel(value_t l)
{
    value_t res = NIL;
    push_reverse_list(l); // 将多个表达式逆序压栈
    while (g_sp != 0) {
        value_t v = pop();
        res = eval(v);    // 依次求值，最后一个返回值
    }
    return res;
}

```

`push_reverse_list` 递归地将列表逆序压入栈：(a b c) → 栈顶 ...c, b, a, 然后逐个 pop 求值。

1.5 求值核心： `eval_sexp`

整个解释器的中枢，采用**基于 goto 的尾调用优化**模式：

```

static value_t eval_sexp(value_t sexp, bool noeval)
{
    int ss = g_sp, ee = g_env_sp;
    int tail_macro = 1;
    value_t res = NIL;
    bool is_apply = false;

eval_top:                // ← 尾递归跳转目标
    switch (type_of(sexp)) {
    case TYPE_SYM:  res = eval_sym(sexp); break;
    case TYPE_LIST: ...求值列表...
    default:        res = sexp;  // 数字等自求值
    }
    goto end;

apply_builtin:          // ← 内建函数应用
apply:                  // ← 用户定义函数应用
end:                    // ← 清理返回
    if (tail_macro && !noeval) tail_eval(res);
    restore_stack(ss);
    env_restore_stack(ee);
    return res;
}

```

调度规则：

1. **符号** → `eval_sym` (查环境栈 → 全局绑定)
2. **列表** → 先求值 `car` 得函数，再分派到 `apply_builtin` 或 `apply`
3. **数字等** → 自求值

`noeval` 参数控制是否进行宏展开 (`expand` 调用 `eval_sexp(v, true)`) 。

2. 核心机制详解

2.1 尾调用优化： `tail_eval` 宏

```
#define tail_eval(exp) do { \
    sexp = (exp);          \
    restore_stack(ss);      \
    goto eval_top;         \
} while(0);
```

不递归，而是复用当前栈帧：

1. `restore_stack(ss)` 弹出当前调用产生的临时值，恢复到进入时的栈指针
2. `sexp = (exp)` 设置新的求值目标
3. `goto eval_top` 重新进入求值主循环

这实现了**尾递归消除**——理论上无限深的尾递归也不会栈溢出。

2.2 尾上下文传递： `tail_macro`

```
int tail_macro = 1;
```

`tail_macro` 是一个计数器，控制尾上下文传播的深度：

- 初始化 `= 1`：当前处于尾上下文
- `B_COND` 匹配分支时： `++tail_macro`
- 函数体最后一个表达式： `tail_macro++`
- 宏展开时： `tail_macro += 2`
- `end:` 处： `if (tail_macro && !noeval) tail_eval(res)`

为什么宏需要 +2：宏展开的结果是一个新的 S-表达式，需要再经过一次完整求值。+2 确保在

`end:` 处 `tail_macro` 减到 1 后仍然满足 `> 0`，触发 `tail_eval`，二轮求值后减到 0 终止。

2.3 列表求值： `TYPE_LIST` 分支

```

case TYPE_LIST:
    push(tail(sexp));          // args = 参数列表, 保存到栈
    fun = eval(head(sexp)); // 求值 car 得到函数对象

apply_top:                    // apply 也跳转到此
    if (type_of(fun) == TYPE_BUILTIN) goto apply_builtin;
    if (type_of(fun) == TYPE_LIST) { // 用户定义的 fn/macro
        args = head(tail(fun));      // 参数名列表
        body = tail(tail(fun));      // 函数体
        goto apply;
    }
    error("Applying not a function");

```

两步求值：先求值 car（函数位置），再根据函数类型走不同分支。

apply_top 是 F_APPLY 的入口：is_apply = true; goto apply_top;

2.4 内建函数应用：apply_builtin

```

apply_builtin:
    code = builtin_val(fun)->code;
    args = pop();
    if (code >= F_ADD) {          // F_ 前缀：需要求值参数
        push_reverse_list(args);
        if (!is_apply) {
            for (int i = g_sp - 1; i >= ss; ++i) {
                g_stack[i] = eval(g_stack[i]);
            }
        }
        is_apply = false;
    }
    nargs = g_sp - ss;
    switch (code) { ... }

```

B_ 特殊形式一览：

B_COND （条件分支）

```

push_list(args);          // 所有分支 [(c1 e1) (c2 e2) ...] 入栈
for (int i = ss; i < g_sp; ++i) {
    value_t pair = g_stack[i];
    value_t cond = head(pair);      // 条件
    push(head(tail_(pair)));        // 表达式 (tail_eval 目标)
    res = eval(cond);
    if (to_bool(res) != NIL) {
        tail_eval(pop());          // 匹配 → 尾求值表达式，不返回
    }
    pop();                        // 不匹配 → 丢弃表达式
}

```

`to_bool` 定义: `NIL` 和 `EMPTY_LIST` 为假, 其他为真。

B_DEF (定义全局绑定)

```

const char* name = sym_val(head(args))->name;
value_t sym = symbol(name, &symtab);    // 查找或创建符号
res = eval(head(tail_(args)));          // 求值表达式
sym_val(sym)->binding = res;            // 设置符号的全局绑定

```

注意: `def` 创建的是**全局绑定**, 永不进入环境栈。

B_OR / B_AND (短路逻辑)


```
// B_OR: 从左到右求值，遇到真值短路
push_reverse_list(args);
while (g_sp > ss) {
    res = eval(pop());
    if (res != NIL && res != EMPTY_LIST) break;
}

// B_AND: 遇到假值短路
push_reverse_list(args);
while (g_sp > ss) {
    res = eval(pop());
    if (res == NIL) break;
}
```

B_FN / B_MACRO (创建闭包) ——关键

```

case B_MACRO:
case B_FN:
{
    int ee1 = g_env_sp;
    body = tail(args);          // 函数体
    args = head(args);          // 参数名列表
    push(args);

    // 将参数名推入环境栈两次，实现 shadowing
    // 这样 copy_body 时参数名在环境栈中找到自身（不替换）
    if (!(code == B_FN && noeval)) {
        for (value_t h = args; h != EMPTY_LIST; h = tail(h)) {
            if (head(h) != REST) {
                env_push(head(h));    // value 占位
                env_push(head(h));    // symbol 占位
            }
        }
    }

    body = copy_body(body);      // ★ 闭包捕获：深拷贝 + 变量替换
    args = pop();
    res = cons_(code == B_FN ? FN : MACRO, cons_(args, body));
    env_restore_stack(ee1);
}

```

关键细节：参数名被推入环境栈两次（`env_push` 两次相同的符号），这样 `copy_body` 在遇到参数名时会在环境中查到其自身（`g_env_stack[i-1] == g_env_stack[i]`），替换为自身——相当于**不替换**。这意味着函数体内引用参数名不会被环境中的其他同名绑定干扰。

B_QUOTE

```

assert_nargs(0);
res = head(args); // 直接返回参数，不求值

```

B_DO（顺序执行）

```

push_reverse_list(args);
while (g_sp > ss) {
    res = eval(pop());    // 依次求值，最后一个作为结果
}

```

F_APPLY

```

fun = pop();           // 从栈顶取出函数对象
is_apply = true;       // 标记：参数已全部求值
goto apply_top;        // 复用函数调度

```

is_apply 防止 apply_builtin 中再次求值参数。

2.5 用户函数应用：apply 分支

```
apply:
    funtype = head(fun);           // FN 或 MACRO
    value_t list = pop();          // 实参列表（已从栈弹出）
    push(body);                    // 保存函数体
    push(args);                    // 保存形参列表

    int ss0 = g_sp;
    push_list(list);               // 实参逐个入栈
    for (int i = ss0; i < g_sp; ++i) {
        if (funtype != MACRO && !is_apply) {
            g_stack[i] = eval(g_stack[i]); // 函数实参求值
        }
        // 宏：实参不求值，保持原样
    }

    args = g_stack[ss0 - 1];       // 取回形参列表
    env_restore_stack(ee);         // 恢复环境栈到调用前

    prepare_env(args, ss0);        // 将实参绑定到形参，入环境栈

    args = pop();                  // 弹出形参
    body = pop();                  // 弹出函数体

    push_reverse_list(body);       // 函数体表达式逆序压栈
    while (g_sp > ss) {
        if (g_sp == ss + 1) {      // 最后一个表达式
            // 设置尾上下文 → tail_eval
            tail_eval(pop());
        } else {
            res = eval(pop());      // 非末表达式，普通求值
        }
    }
}
```

宏 (MACRO) vs 函数 (FN) 关键差异:

特性	FN	MACRO
实参求值	是	否

特性	FN	MACRO
tail_macro 递增	+1	+2
展开后处理	直接返回	结果重新 tail_eval 进入求值循环

2.6 end: 与尾调用传播

```
end:
    if (tail_macro && !noeval) {
        tail_eval(res);    // 重新进入求值循环
    }
    restore_stack(ss);
    env_restore_stack(ee);
    return res;
```

tail_macro 递减链:

```
调用函数体最后表达式 → tail_macro=2 → end: tail_eval
→ eval_top: tail_macro=1 → 结果走到 end: tail_eval
→ eval_top: tail_macro=0 → 不再尾求值，正常返回
```

宏展开时额外 +2，确保结果 S-表达式经历完整求值。

3. 辅助函数详解

3.1 eqp —— 相等判定

```

static value_t eqp(value_t v1, value_t v2)
{
    if (type_of(v1) != TYPE_LIST || type_of(v2) != TYPE_LIST) {
        return v1 == v2 ? T : NIL;
    }

    while (v1 != EMPTY_LIST && v2 != EMPTY_LIST) {
        if (eqp(head(v1), head(v2)) == NIL) {
            return NIL;
        }
        v1 = tail(v1);
        v2 = tail(v2);
    }
    return v1 == v2 ? T : NIL;
}

```

递归比较两个值的结构相等性： - **非列表**：指针/值直接比较 - **列表**：递归比较每个元素，长度不同则返回假

注意：原代码第 399 行为 `head(v1) != head(v2)`（指针比较），这是一个 bug——嵌套列表比较时应当递归调用 `eqp`，而不是做浅指针比较。

3.2 `copy_body` ——闭包变量捕获

在 `B_FN` / `B_MACRO` 创建闭包时调用，做两件事： 1. **深拷贝**函数体（防止后续修改影响闭包） 2. **变量替换**：将环境中绑定的自由变量替换为实际值

```

static value_t copy_body(value_t body)
{
    int ss = g_sp;
    switch (type_of(body)) {
    case TYPE_LIST:
        // 子分支 A: 空列表或 quoted → 直接返回
        if (body == EMPTY_LIST || head(body) == QUOTE) {
            return body;
        }
        // 子分支 B: 宏展开
        if (is_sym(head(body))) {
            push(body);
            value_t tmp = eval(head(body));
            body = pop();
            if (is_list(tmp) && head(tmp) == MACRO) {
                body = expand(body);
                return copy_body(body);
            }
        }
        // 子分支 C: 空环境短路优化
        if (g_env_sp == 0) return body;
        // 子分支 D: 递归深拷贝
        push_list(body);
        for (int i = ss; i < g_sp; ++i) {
            g_stack[i] = copy_body(g_stack[i]);
        }
        return pop_list(ss);

    case TYPE_SYM:
        // 在环境栈中查找符号 → 替换为值
        for (int i = g_env_sp - 1; i > 0; i -= 2) {
            if (g_env_stack[i] == body) {
                return g_env_stack[i - 1];
            }
        }
        return body; // 自由变量, 保持符号
    }
}

```

```
    default:
        return body;  // 数字、内置函数等
    }
}
```

四个子分支：

分支	条件	行为	原因
A	EMPTY_LIST 或 QUOTE	直接返回	quoted 数据中的符号不是变量引用
B	car 是宏	展开后再递归	宏展开前的符号不应被捕获
C	g_env_sp == 0	直接返回	无词法绑定，无需替换
D	其他列表	递归拷贝每个元素	标准深拷贝流程

捕获示例：

```
(let ((x 42)) (fn (y) (+ x y)))
```

创建闭包时 `copy_body` 处理 `(+ x y)`：
`- +` $\rightarrow$ `default` $\rightarrow$ `+ - x` $\rightarrow$ `TYPE_SYM`，环境栈查到 `42` $\rightarrow$ `42` **(被替换)** `- y` $\rightarrow$ `TYPE_SYM`，环境栈无 $\rightarrow$ `y` **(保持符号)**
结果闭包体：`(+ 42 y)`，`x` 已被"捕获"为值 42。

3.3 `prepare_env` ——形参绑定

在函数调用时，将实参按形参列表绑定到环境栈：


```

static void prepare_env(value_t args, int ss)
{
    int sp = ss;
    if (is_list(args)) {          // 固定参数: (x y z)
        for (value_t h = args; h != EMPTY_LIST; h = tail(h)) {
            if (head(h) != REST) {
                env_push(g_stack[ss]); // 实参值
                env_push(head(h));     // 形参名
                ++ss;
            } else {
                args = head(tail(h)); // 变参符号
                break;
            }
        }
    }
    if (!is_list(args)) {        // 变参: & rest
        env_push(pop_list(ss));   // 剩余实参打包为列表
        env_push(args);          // 形参名
    }
    restore_stack(sp);
}

```

示例： (fn (x & y) ...) 调用 (f 1 2 3)： - 循环绑定 $x \leftarrow 1$ - 遇到 &，剩余实参 [2, 3] 打包为 (2 3) - 变参绑定： $y \leftarrow (2\ 3)$

3.4 eval_sym —— 符号求值

```

static value_t eval_sym(value_t v)
{
    for (int i = g_env_sp - 1; i >= 0; i -= 2) {
        if (g_env_stack[i] == v) {
            return g_env_stack[i - 1]; // 环境栈中找到
        }
    }
    return sym_val(v)->binding;      // 回退到全局绑定
}

```

两层查找：

1. **环境栈**（词法作用域）：从内向外（`i -= 2`），找到最近绑定返回
2. **全局符号表**（动态后备）：未找到时查符号的 `binding` 字段

4. 典型例子跟踪

4.1 定义函数： `(def add (fn (x y) (+ x y)))`

1. `eval_toplevel`: 列表 `(def add (fn (x y) (+ x y)))`

2. `TYPE_LIST`:

`args = (add (fn (x y) (+ x y)))`

`fun = eval(def) → BUILTIN(B_DEF)`

3. `B_DEF`:

`name = "add"`

求值 `rhs`: `eval((fn (x y) (+ x y)))`

→ 4. 内层 `eval`:

`args = ((x y) (+ x y))`

`fun = eval(fn) → BUILTIN(B_FN)`

→ 5. `B_FN`:

`body = ((+ x y)), args = (x y)`

推入 `x, y` 到环境栈（`shadow` 自身）

`copy_body((+ x y))`:

`+ → + (default)`

`x → TYPE_SYM`, 环境查到 `x → x`（阴影自身）

`y → TYPE_SYM`, 环境查到 `y → y`

→ `(+ x y)`（内容未变）

结果: `(FN (x y) (+ x y))`

6. `sym_val(add) → binding = (FN (x y) (+ x y))`

4.2 调用函数： `(add 3 4)`

```

1. TYPE_LIST:
    args = (3 4)
    fun = eval(add) → 查全局 → (FN (x y) (+ x y))

2. apply:
    funtype = FN
    list = (3 4)
    实参求值: eval(3)=3, eval(4)=4

3. prepare_env:
    x ← 3, y ← 4 入环境栈

4. body = ((+ x y))
    push_reverse → 栈: [(+ x y)]
    g_sp == ss+1 → tail_eval((+ x y))

5. TYPE_LIST:
    args = (x y)
    fun = eval(+) → BUILTIN(F_ADD)

6. F_ADD:
    求值参数: eval(x) → 环境栈 → 3
               eval(y) → 环境栈 → 4
    sum = 3 + 4 = 7

7. end: tail_macro=1 → tail_eval(7)
8. eval_top: TYPE_NUM → res = 7 → end → 返回 7

```

4.3 闭包捕获: let 展开后

```
(def foo (let ((x 42)) (fn (y) (+ x y))))
```

let 展开为: (def foo ((fn (x) (fn (y) (+ x y))) 42))

1. 外层 fn 调用, x=42 入环境栈
2. 内层 B_FN 创建闭包 (fn (y) (+ x y)) 时:
 copy_body((+ x y)):
 + → +
 x → TYPE_SYM, 环境栈查到 42 → 42 ← 捕获!
 y → TYPE_SYM, 环境栈无 → y
 结果: (+ 42 y)
3. foo 绑定到 (FN (y) (+ 42 y))
4. (foo 5) → (+ 42 5) → 47 ✓ 闭包正常工作

为什么 x 被捕获为 42 而不是 x?

调用外层 fn 时 prepare_env 推入的是 [42, x] (值后跟符号)。copy_body 找到 x 时返回 g_env_stack[i-1], 即 42 ——实际的数字值。

4.4 宏展开

```
(defmacro unless (c & b)
  (list 'if (list 'not c) (cons 'do b)))

(unless #t (print 1) (print 2))
```

1. `eval(unless #t (print 1) (print 2))`
2. `fun = eval(unless) → 查全局 → MACRO`
3. `apply:`
`funtype = MACRO`
 实参不求值: `list = (#t (print 1) (print 2))`
`prepare_env: c ← #t, b ← ((print 1) (print 2))`
4. 宏体求值:
`(list 'if (list 'not c) (cons 'do b))`
`→ 展开为 (if (not #t) (do (print 1) (print 2)))`
5. `end: tail_macro = 1(初始化) + 2(宏) = 3`
`→ tail_eval(展开结果)`
6. 二轮求值: `(if (not #t) (do (print 1) (print 2)))`
`B_COND: (not #t) → NIL, 走 else`
`→ (do (print 1) (print 2)) 执行`
7. 输出: `1\n2\n`

5. `system.lsp` 解读

`system.lsp` 是一个 "引导库" (bootstrap library), 基于极小核心原语提供了常用语法和函数。

5.1 文件全览

```

(def else '#t)

(def list (fn args args))

(def atom? (fn (x) (not (list? x))))

(def splice-body
  (fn (body)
    (cond ((atom? body) body)
          ((= (tail body) '()) (head body))
          (else (cons do body)))))

(defmacro defmacro (name args & body)
  (list 'def name (list 'macro args (splice-body body))))

(defmacro defun (name args & body)
  (list 'def name (list 'fn args (splice-body body))))

(defun null? (l)
  (= l '()))

(defmacro if (c t & e)
  (cond ((null? e) (list 'cond (list c t)))
        (else (list 'cond (list c t) (list 'else (head e))))))

(defun map (f lst)
  (if (null? lst) lst
      (cons (f (head lst)) (map f (tail lst)))))

(defmacro let (binds & body)
  (cons (list 'fn (map head binds) (splice-body body))
        (map snd binds)))

(def fst head)

(defun snd (l)
  (head (tail l)))

```

```

(defun fold (f s l)
  (if (null? l) s
      (fold f (f s (head l)) (tail l))))

(defun fold1 (f l)
  (fold f (head l) (tail l)))

(defun any? (p l)
  (fold1 or (map p l)))

(defun append (l1 l2)
  (if (null? l1) l2
      (cons (head l1) (append (tail l1) l2))))

(defmacro quasiquote (e)
  (qq-expand e))

(defun qq-expand (x)
  (if (list? x)
      (cond ((null? x) (list 'quote '()))
            ((= (head x) 'unquote) (snd x))
            ((= (head x) 'quasiquote)
             (qq-expand (qq-expand (snd x))))
            (else (list 'append
                        (qq-expand-list (head x))
                        (qq-expand (tail x)))))
      (list 'quote x)))

(defun qq-expand-list (x)
  (if (list? x)
      (cond ((null? x) (list 'quote (list '())))
            ((= (head x) 'unquote) (list 'list (snd x)))
            ((= (head x) 'unquote-splicing) (snd x))
            ((= (head x) 'quasiquote)
             (qq-expand-list (qq-expand (snd x))))
            (else (list 'list
                        (qq-expand-list (head x))
                        (qq-expand-list (tail x)))))
      (list 'quote x)))

```

```

      (list 'append
            (qq-expand-list (head x))
            (qq-expand (tail x))))))
  (list 'quote (list x))))

(defun zip l
  (if (any? null? l) '()
      (cons (map head l) (apply zip (map tail l)))))

```

5.2 分层解读

第 1 层：基础常量与构造

```

(def else '#t)           ; cond 的 else 分支标记
(def list (fn args args)) ; 变参构造列表

```

`(def list (fn args args))` 利用了 `fn` 的变参绑定——参数名（不加括号）绑定到所有实参组成的列表，函数体直接返回这个列表。

```

(list 1 2 3) → args = (1 2 3) → (1 2 3)
(list)      → args = ()    → ()

```

对比需要递归的写法，这个定义极简。

第 2 层：谓词

```

(def atom? (fn (x) (not (list? x)))) ; 是否为原子
(defun null? (l) (= l '()))          ; 是否为空列表

```

第 3 层：宏基础设施 `splice-body`

```

(def splice-body
  (fn (body)
    (cond ((atom? body) body)           ; 原子 → 直接返回
          ((= (tail body) '()) (head body)) ; 单表达式 → 直接返回
          (else (cons do body))))       ; 多表达式 → 包装为 (do ...)

```


自动处理函数/宏体：单表达式直接使用，多表达式包裹 `do`。

第 4 层：定义宏

```
(defmacro defmacro (name args & body)
  (list 'def name (list 'macro args (splice-body body))))

(defmacro defun (name args & body)
  (list 'def name (list 'fn args (splice-body body))))
```

`defun` 展开为 `(def name (fn args (splice-body body)))`，`defmacro` 类似。

第 5 层：条件宏 `if`

```
(defmacro if (c t & e)
  (cond ((null? e) (list 'cond (list c t)))
        (else (list 'cond (list c t) (list 'else (head e))))))
```

将 `if` 展开为 `cond`：无 `else` → 单分支 `cond`；有 `else` → 双分支 `cond`。

第 6 层：核心高阶函数

```

(defun map (f lst)
  (if (null? lst) lst
      (cons (f (head lst)) (map f (tail lst)))))

(defun fold (f s l)
  (if (null? l) s
      (fold f (f s (head l)) (tail l))))

(defun fold1 (f l)
  (fold f (head l) (tail l)))

(defun any? (p l)
  (fold1 or (map p l)))

(defun append (l1 l2)
  (if (null? l1) l2
      (cons (head l1) (append (tail l1) l2))))

```

第 7 层: `let` 宏

```

(defmacro let (binds & body)
  (cons (list 'fn (map head binds) (splice-body body))
        (map snd binds)))

```

展开为立即调用的 lambda (Scheme 风格) :

```

(let ((x 1) (y 2)) (+ x y))
;; 展开为:
((fn (x y) (+ x y)) 1 2)

```

`map head binds` $\rightarrow$ `(x y)` (形参列表) `map snd binds` $\rightarrow$ `(1 2)` (实参列表)

第 8 层: 准引用系统

```

(defmacro quasiquote (e) (qq-expand e))

```

完整实现 quasiquote 展开，支持： - `(a ,b c) → (list 'a b 'c) - `(a ,@b c) → (append (list 'a) b (list 'c)) - 嵌套 ` 的正确处理

第 9 层： zip

```
(defun zip l
  (if (any? null? l) '()
      (cons (map head l) (apply zip (map tail l))))))
```

矩阵转置： (zip '(a b) '(1 2)) → ((a 1) (b 2))

5.3 设计特点

特性	说明
极小核心	仅依赖 def / fn / macro / cond / head / tail / list? / = / quote / & 等原语
自举风格	defun / if / let 都用宏实现，解释器无需内置
准引用	完整 quasiquote，是宏编写的关键基础设施
函数式	map / fold / any? / zip / append 构成基础库
简洁	83 行实现一个可用的 Lisp 标准库子集

6. 设计要点总结

设计决策	作用
Tagged pointer	数字零分配，类型判断仅检查低 2 位，无额外内存开销
值栈 + 环境栈分离	值栈管理求值中间结果，环境栈管理变量查找，职责清晰
深拷贝闭包	copy_body 在闭包创建时替换自由变量，运行时无需访问环境栈
tail_eval + goto	基于 goto 的尾递归消除，循环代替递归，无栈溢出风险
tail_macro 计数器	控制宏展开链深度，确保展开结果被二次求值
特殊形式 vs 函数	B_ 前缀自主控制参数求值时机，F_ 前缀自动求值全部参数
参数 shadow 技巧	参数名推入环境栈两次，使 copy_body 将参数替换为自身——巧妙避开参数捕获
quote 保护	copy_body 在遇到 QUOTE 时直接返回，不遍历内部，避免数据中的符号被误替换

本文档由 *Lisp* 解释器项目会话讲解整理生成